BCP

PROGRAMA · TRAIN THE TRAINERS

# Formación *AI Engineering.*

Prompting, contexto, seguridad, agent skills, templates del SDLC, tool calling con MCP, optimización y casos E2E (Hackathon)

| DURACIÓN | FORMATO | COBERTURA |
| --- | --- | --- |
| **8 módulos × 5 h · 40 h** | **Train the Trainers** | **5 grupos × 5 personas** |



© 2026 NTT DATA, Inc.

**Recorrido completo** — Vista del programa

# Ruta de aprendizaje, *5 días.*

Cinco jornadas de 09:00 a 18:00. Examen de entrada el Día 1; Gate G1 y Gate G2 al cierre de los Días 2 y 4.

## 1 Día

### Prompting & Secure AI Engineering

- Prompting + Context Engineering
- Secure AI Engineering
- Optimización + Refactor
- Docs + Unit Testing

**EXAMEN DE ENTRADA**
Día 1 · 09:30–10:00

## 2 Día

### Agent Skills, Templates & MCP

- Anatomía del Agent Skill
- Templates del nuevo SDLC
- Tool Calling
- MCP aplicado al SDLC

**GATE G1 · EVALUACIÓN**
Día 2 · 17:30–18:00

## 3 Día

### SDD + Agentes

- OpenSpec, Spec Kit y Constitution
- Specify → Plan → Tasks → Validate
- Fundamentos de agentes
- Routing, Evaluator y HITL

## 4 Día

### Agentic Engineering in Practice

- CrewAI: agentes por roles
- LangGraph: state y routing
- Harness Engineering
- Guardrails

**GATE G2 · EVALUACIÓN**
Día 4 · 17:30–18:00

## 5 Día

### Hackathon AI Engineering

- Alinear y priorizar
- Construir en dos sprints
- Validar y congelar la versión
- Demo Arena y certificación

© 2026 NTT DATA, Inc.

**Programa** — Horario por jornada

# Cinco jornadas, *un mismo ritmo.*

| | DÍA 1 Prompting & Secure AI | DÍA 2 Skills, Templates & MCP | DÍA 3 SDD + Agentes | DÍA 4 Agentic Engineering | DÍA 5 Hackathon E2E |
|---|---|---|---|---|---|
| 08:45 | 08:45–09:00 Coffee break | 08:45–09:00 Coffee break | 08:45–09:00 Coffee break | 08:45–09:00 Coffee break | 08:45–09:00 Coffee break y registro |
| 09:00 | 09:00–09:15 Introducción BCP | 09:00–09:15 Apertura y repaso Día 1 | 09:00–09:15 Apertura y repaso Día 2 | 09:00–09:15 Apertura y repaso Día 3 | 09:00–09:15 Kick-off: reglas y reto |
| | 09:15–09:45 **Examen de entrada** | 09:15–10:00 S1 · Anatomía del Agent Skill | 09:15–10:00 S1 · SDD: OpenSpec, Spec Kit | 09:15–10:00 S1 · CrewAI: agentes por roles | 09:15–09:45 1 · Alinear |
| 10:00 | 09:45–10:30 M1 · Prompting + Context Eng. | 10:00–10:50 Lab 1 · Skill básico versionado | 10:00–10:50 Lab 1 · Definición técnica + contrato | 10:00–11:00 Lab 1 · Crew API / BE / FE / reviewer | 09:45–10:15 2 · Priorizar |
| | 10:30–11:15 Lab 1 · Reescribir pedidos vagos | 10:50–11:30 S2 · Templates del SDLC | 10:50–11:30 S2 · Specify → Plan → Tasks | 11:00–11:35 S2 · LangGraph: state y routing | 10:15–10:30 Checkpoint 1 · Scope |
| 11:00 | 11:15–11:50 M2 · Secure AI Engineering | 11:30–12:30 Lab 2 · Skill desde template | 11:30–12:30 Lab 2 · Spec-to-tasks | 11:35–12:30 Lab 2 · Graph con estado | 10:30–12:30 3 · Construir · Sprint 1 |
| 12:00 | 11:50–12:30 Lab 2 · Detección y corrección segura | | | | |
| 13:00 | 12:30–13:30 Almuerzo | 12:30–13:30 Almuerzo | 12:30–13:30 Almuerzo | 12:30–13:30 Almuerzo | 12:30–13:30 Almuerzo |
| 14:00 | 13:30–14:15 M3 · Refactor y optimización + demo | 13:30–14:20 S3 · Tool calling + demo | 13:30–14:15 S3 · Fundamentos de agentes | 13:30–14:00 S3 · Harness Engineering | 13:30–14:45 3 · Construir · Sprint 2 |
| | 14:15–15:20 Lab 3 · Refactor guiado con Copilot | 14:20–15:20 S4 · MCP: tools, resources y permisos | 14:15–15:25 Lab 3 · Single-agent | 14:00–15:15 Lab 3 · Harness del workflow | |
| 15:00 | 15:20–16:00 M4 · Docs + unit testing asistido | 15:20–16:00 Lab 3 · Skill + MCP en local | 15:25–16:00 S4 · Routing, Evaluator y HITL | 15:15–16:00 S4 · Guardrails | 14:45–15:30 4 · Validar |
| 16:00 | 16:00–16:15 Coffee break | 16:00–16:15 Coffee break | 16:00–16:15 Coffee break | 16:00–16:15 Coffee break | 15:30–16:00 Checkpoint 2 · Freeze |
| | 16:15–17:45 Lab 4 · Docs y unit tests | 16:15–17:15 Lab 3 (cont.) · Skill + MCP | 16:15–17:45 Lab 4 · Agente + review gate | 16:15–17:10 Lab 4 · Agente seguro | 16:00–16:15 Coffee break |
| 17:00 | | 17:15–17:30 Cierre del Día 2 | | 17:10–17:30 Síntesis · puente al Día 5 | 16:15–17:15 5 · Demo Arena · 2 salas × 5 equipos |
| | | 17:30–18:00 **Gate G1 · Evaluación** | | 17:30–18:00 **Gate G2 · Evaluación** | 17:15–17:30 Deliberación del jurado |
| 18:00 | 17:45–18:00 Cierre · puente al Día 2 | | 17:45–18:00 Cierre · puente al Día 4 | | 17:30–18:00 **Cierre + Certificado** |

Contenido · Laboratorio · Apertura · cierre · Examen y Gates G1 / G2 · Certificación · Coffee · almuerzo

© 2026 NTT DATA, Inc.

BCP

BCP · TRAIN-THE-TRAINERS · DÍA 4

# Agentic AI *en la práctica.*

De los agentes al sistema completo: cómo CrewAI, LangGraph, el harness y los guardrails convierten una idea en una operación bancaria que escala con seguridad.

**AUDIENCIA**

**Train the trainers**

**FORMATO**

**4 sesiones - Teoría + comparativa + demo o práctica guiada en cada bloque.**

**PROPÓSITO**

**De agente a sistema**



© 2026 NTT DATA, Inc.

# ¿Qué es un *agente?*

Un LLM responde. Un agente decide, actúa y repite hasta lograr el objetivo.

**PREGUNTA GUÍA** *¿Qué le agregamos a un modelo para que deje de responder y empiece a actuar?*

## LLM VS. AGENTE

**LLM** Genera una respuesta a partir de un prompt.




**AGENTE** Usa un modelo para decidir, usa herramientas, mantiene contexto y repite el ciclo.




↻ *El ciclo continúa hasta cumplir el criterio de éxito.*

## UN AGENTE =

- ✓ **Modelo (LLM)**
  Razona y decide la siguiente acción
- ✓ **Instrucciones**
  Rol, reglas y formato de salida
- ✓ **Contexto**
  Lo que el modelo ve en cada paso
- ✓ **Herramientas (Tools)**
  El entorno las ejecuta: APIs, MCP
- ✓ **Bucle de decisión (Loop)**
  Razona, actúa y observa hasta la salida
- ✓ **Estado (State)**
  Qué se hizo y en qué paso va

**LO CLAVE** **Un LLM es una pieza; un agente es el sistema completo que decide, actúa y mantiene estado.**

# El bucle *agéntico.*

Seis pasos que se repiten, alimentados por contexto, herramientas y estado.




TRES PIEZAS ALIMENTAN CADA VUELTA

**C** **Contexto**

Lo que el modelo ve en cada vuelta: instrucciones, historial, datos recuperados, resultados y políticas.

**T** **Tool calling**

El modelo pide la herramienta (nombre + parámetros); el entorno la ejecuta y devuelve el resultado.

**E** **Estado**

Lo que el sistema recuerda entre vueltas: qué se hizo, en qué paso va y el tope de pasos.

LO CLAVE **El agente no termina al responder: termina cuando cumple el criterio de éxito o llega a su tope de pasos.**

NTT DATA

BCP

ORQUESTACIÓN MULTIAGENTE · LABORATORIO

# CrewAI y *LangGraph.*



Dos formas de orquestar agentes: equipos por roles con CrewAI y flujos con estado y control explícito con LangGraph. Cuándo usar cada una en el SDLC.

| AUDIENCIA | PLATAFORMA | FORMATO |
| --- | --- | --- |
| **Líderes de desarrollo BCP** | **GitHub Copilot Enterprise** | **Concepto + laboratorio** |



© 2026 NTT DATA, Inc.

# CrewAI — Multi-Agent

Cuando una sola identidad no alcanza, varios agentes con roles, herramientas y backstories empiezan a hacer el trabajo que el prompting solo no puede.

**CLAVE:** Anatomía de un crew · tres patrones de orquestación · HITL manual · un caso aplicado a la operación BCP.

# CrewAI, *agentes con roles.*

CrewAI es un **framework de Python** para orquestar varios agentes como un equipo que se reparte una tarea compuesta.

QUÉ ES

## Un equipo, no una sola voz.

Cada agente tiene **rol, objetivo y backstory**; cada tarea, **un responsable y un resultado esperado**. El crew los pone a trabajar juntos: el resultado del primero alimenta al segundo y el manager coordina, valida o escala.

POR QUÉ IMPORTA

## Tres razones para usar un crew.

**1 · Especialización:** cada agente se enfoca en su rol.

**2 · Trabajo en equipo:** las tareas compuestas se reparten.

**3 · Control humano:** un manager coordina y un paso HITL revisa.

UNA FRASE

## CrewAI *no* es un chatbot.
## Es el equipo de trabajo que tiene un chatbot en cada silla.

Cuando el problema es **una pregunta**, prompting basta. Cuando el problema es **una decisión compuesta** — interpretar, investigar, contrastar, redactar — los roles empiezan a pagar.

**PUENTE** CrewAI es una opinión sobre cómo organizar el sistema. La siguiente lámina lo desarma.

# CrewAI: *equipos de agentes.*

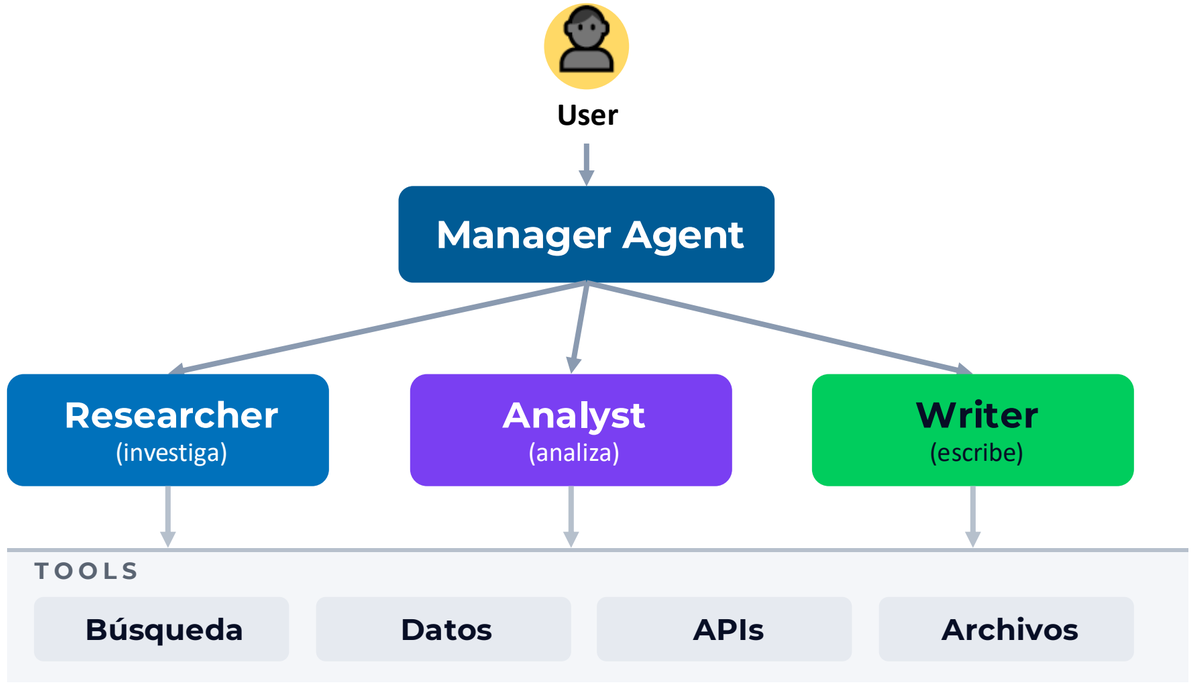
Varios agentes con roles definidos que colaboran para completar una tarea.

**PREGUNTA GUÍA** *¿Qué rol debería tener cada agente de tu equipo y quién coordina?*

## CONCEPTOS CLAVE

1. **Agent**
   rol, goal, backstory
2. **Task**
   tarea específica
3. **Crew**
   equipo de agentes
4. **Process**
   sequential / hierarchical
5. **Delegation**
   entre agentes

## ARQUITECTURA




**LO CLAVE** **Un crew es un equipo: un manager coordina y cada agente aporta su rol, sus tools y su tarea.**

# Tres patrones, *un HITL al final.*

El proceso es la decisión política del crew: define **quién manda, cuánto se delega** y **dónde vuelve a aparecer la persona**.

## A → B → C

### Sequential

Una tarea termina y la siguiente empieza con **su resultado**. Predecible y fácil de testear.

**SE ELIGE CUANDO**

El trabajo es lineal y el orden no se negocia.

**EJEMPLO EN BANCA**

Conciliación de pagos al cierre: un agente extrae, otro cruza con el core y otro reporta las diferencias.

**RIESGO**

Falla en cascada: si A se equivoca, todos heredan.

## M

### Hierarchical

Un **manager** recibe la consulta, la descompone, asigna y revisa antes de devolver.

**SE ELIGE CUANDO**

Alguien debe decidir qué subtarea va primero.

**EJEMPLO EN BANCA**

Incidente en banca móvil: el manager asigna logs, métricas y cambios recientes según lo que aparece.

**RIESGO**

El manager se vuelve el cuello de botella.

## A 🤝 B 🤝 C

### Consensus

Varios agentes proponen **en paralelo** y un agregador decide.

**SE ELIGE CUANDO**

Hay varias respuestas válidas y conviene comparar.

**EJEMPLO EN BANCA**

Riesgo de un crédito: tres agentes analizan por separado y un agregador compara; decide una persona.

**RIESGO**

Multiplica el coste por N.

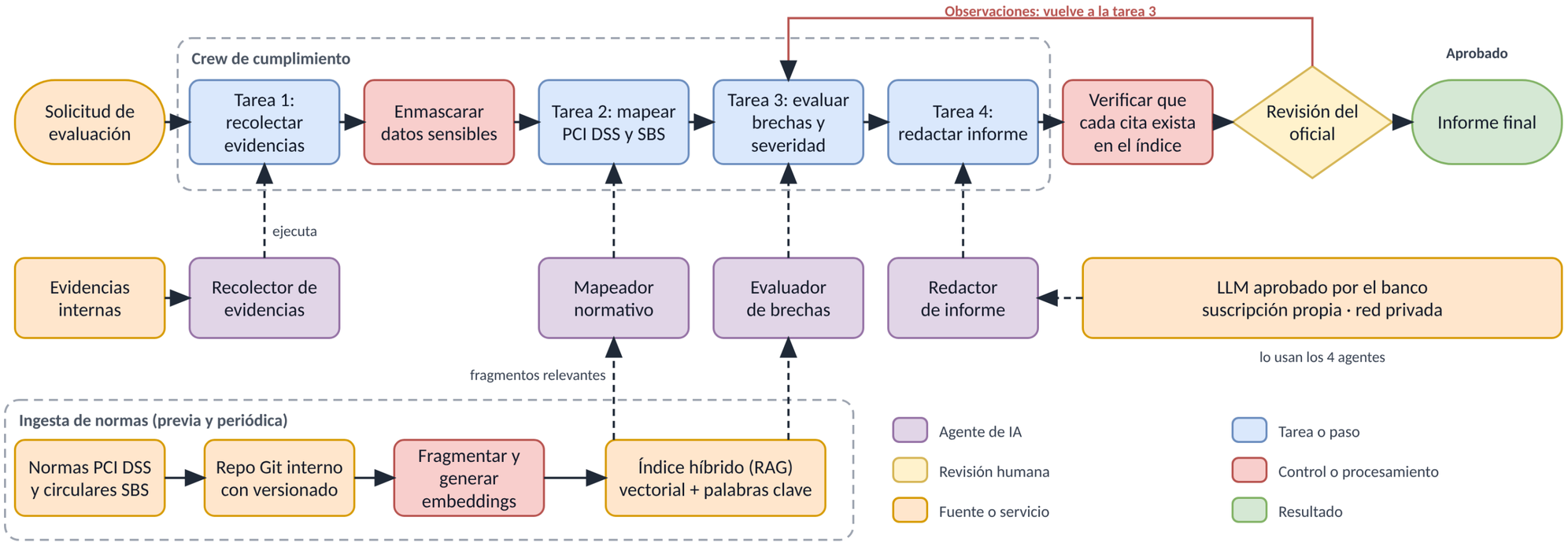
**Y DESPUÉS, EL HUMANO**

Cualquiera de los tres patrones vuelve al humano cuando aparece un caso sensible: CrewAI permite un nodo **HITL (Human-In-The-Loop)** antes del cierre. Lo vemos en detalle en la S4.

# Caso: *cumplimiento normativo.*

Cuatro agentes especializados evalúan PCI DSS y SBS con RAG; el oficial de cumplimiento revisa antes de cerrar.

## CASO · EVALUACIÓN DE CUMPLIMIENTO PCI DSS Y SBS CON RAG




**LO CLAVE** **El valor está en la especialización: cada agente aporta su rol, el crew coordina y una persona aprueba el informe.**

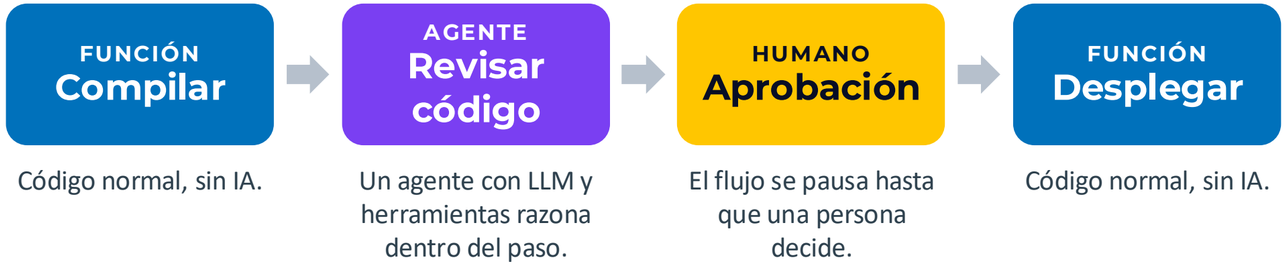
# LangGraph — State & Graph

Cuando CrewAI describe *quién* hace qué, LangGraph describe **cómo** fluye la información entre ellos — y dónde se decide, persiste y aprueba.

# LangGraph: *un pipeline para IA.*

Tú dibujas el mapa: nodos que hacen el trabajo, aristas que fijan el orden y un estado compartido entre pasos.

## UN PIPELINE EN EL QUE ENTRAN LOS AGENTES




Código normal, sin IA.

Un agente con LLM y herramientas razona dentro del paso.

El flujo se pausa hasta que una persona decide.

Código normal, sin IA.

**Estado compartido: viaja de paso en paso y se guarda en cada uno (checkpoint).**

**Arista condicional**
Si la revisión falla, el flujo vuelve al paso anterior, con tope de reintentos.

### LA IDEA CLAVE

**El grafo no decide cómo piensa el agente: decide cuándo corre, con qué estado entra y a dónde va su resultado.**

## CREWAI VS. LANGGRAPH

**CrewAI**
**Autonomía con roles**
Un agente orquestador decide, en el momento, a quién despertar.

**LangGraph**
**Control explícito del flujo**
El diseñador deja el recorrido dibujado de antemano; el agente vive dentro de un nodo.

### PARA ELEGIR

**¿Necesitas colaboración entre roles o control, trazabilidad y aprobaciones paso a paso?**

**LO CLAVE** — **En CrewAI un agente orquestador decide a quién despertar; en LangGraph el diseñador dibuja el recorrido y el agente vive en un nodo.**

# Cuatro conceptos *para leer un grafo.*

Con estos cuatro ya lees un grafo de LangGraph. El resto son detalles que el instructor amplía si hace falta.

01 · STATE

### La memoria compartida

La estructura de datos que viaja entre nodos. Cada nodo la lee y la actualiza.

02 · NODE

### Una caja del pipeline

Puede ser una función normal o un agente completo, con su propio bucle y herramientas.

03 · CONDITIONAL EDGE

### La decisión explícita

Una función mira el estado y elige a qué nodo ir. Aquí vive la lógica de negocio.

04 · CHECKPOINT + PAUSA

### Guardar y pausar

Guarda el estado en cada paso para reanudar y auditar; un nodo puede esperar aprobación humana.

CUÁNDO CONVIENE LANGGRAPH

## Cuando el control del flujo importa más que la libertad del agente.

**1 · Aprobaciones humanas**
Revisión de riesgo o de un cambio antes de pasar a producción: el flujo se pausa y se reanuda.

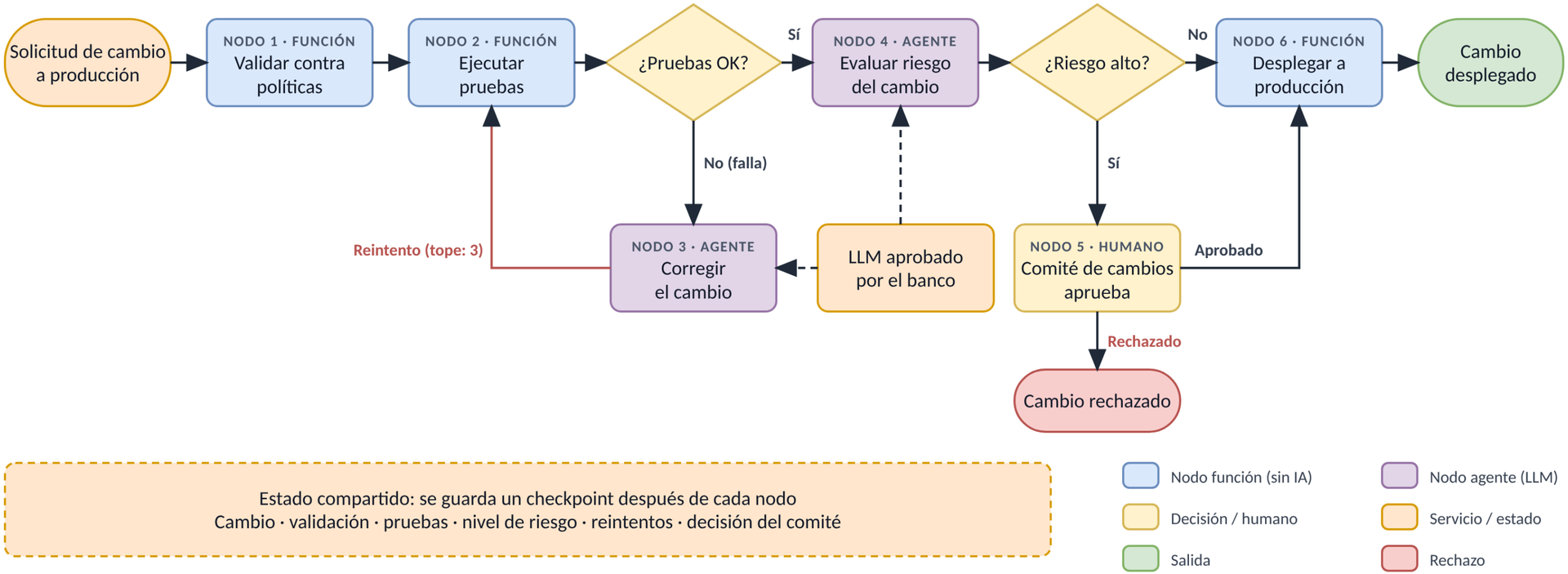
**2 · Trazabilidad**
Procesos que se auditan, se reanudan o se repiten: el estado se guarda en cada paso.

**3 · Ciclos con tope**
Reintentos con condición de salida y presupuesto acotado: el sistema no corre sin fin.

# Caso: *aprobar un cambio a producción.*

Pasos fijos, un agente que corrige y otro que evalúa el riesgo, y una pausa humana con rastro completo.

## CASO · APROBACIÓN DE UN CAMBIO A PRODUCCIÓN EN EL CORE BANCARIO




**LO CLAVE** **El valor está en el control: recorrido visible, reintentos con tope y una aprobación humana que deja rastro de auditoría.**

# Elige por la forma *del problema.*

La herramienta sigue al trabajo, no al revés.

01 · CREWAI

### Un equipo con roles

- Trabajo compuesto que se reparte por especialidad.
- El equipo decide cómo colaborar.
- El resultado es un entregable: informe, revisión, análisis.

02 · LANGGRAPH

### Un flujo con control

- Recorrido explícito y auditable.
- Aprobaciones humanas y reintentos con tope.
- El proceso se pausa y se reanuda.

03 · UN SOLO AGENTE

### El caso simple

- Una tarea acotada, con herramientas.
- Sin reparto de roles ni aprobaciones.
- Empieza aquí; sube de nivel si lo necesitas.

EN BANCA

## Mismo banco, problemas distintos: la herramienta sigue a la forma del problema.

**Cumplimiento PCI DSS y SBS → CrewAI**
Cuatro agentes especializados producen un informe con citas; una persona lo revisa.

**Cambio a producción en el core → LangGraph**
Pruebas, riesgo y comité con pausa humana; reintentos con tope y rastro de auditoría.

**Se pueden combinar**
Un crew de agentes puede vivir dentro de un nodo de un grafo.

NTT DATA

BCP

— INGENIERÍA DE AGENTES · FUNDAMENTOS

# Harness *Engineering.*



Lo que rodea al modelo y lo vuelve confiable: herramientas, contexto, controles y evaluación para llevar agentes del demo a la operación.

| AUDIENCIA | PLATAFORMA | FORMATO |
| --- | --- | --- |
| **Líderes de desarrollo BCP** | **GitHub Copilot Enterprise** | **Concepto + laboratorio** |



© 2026 NTT DATA, Inc.

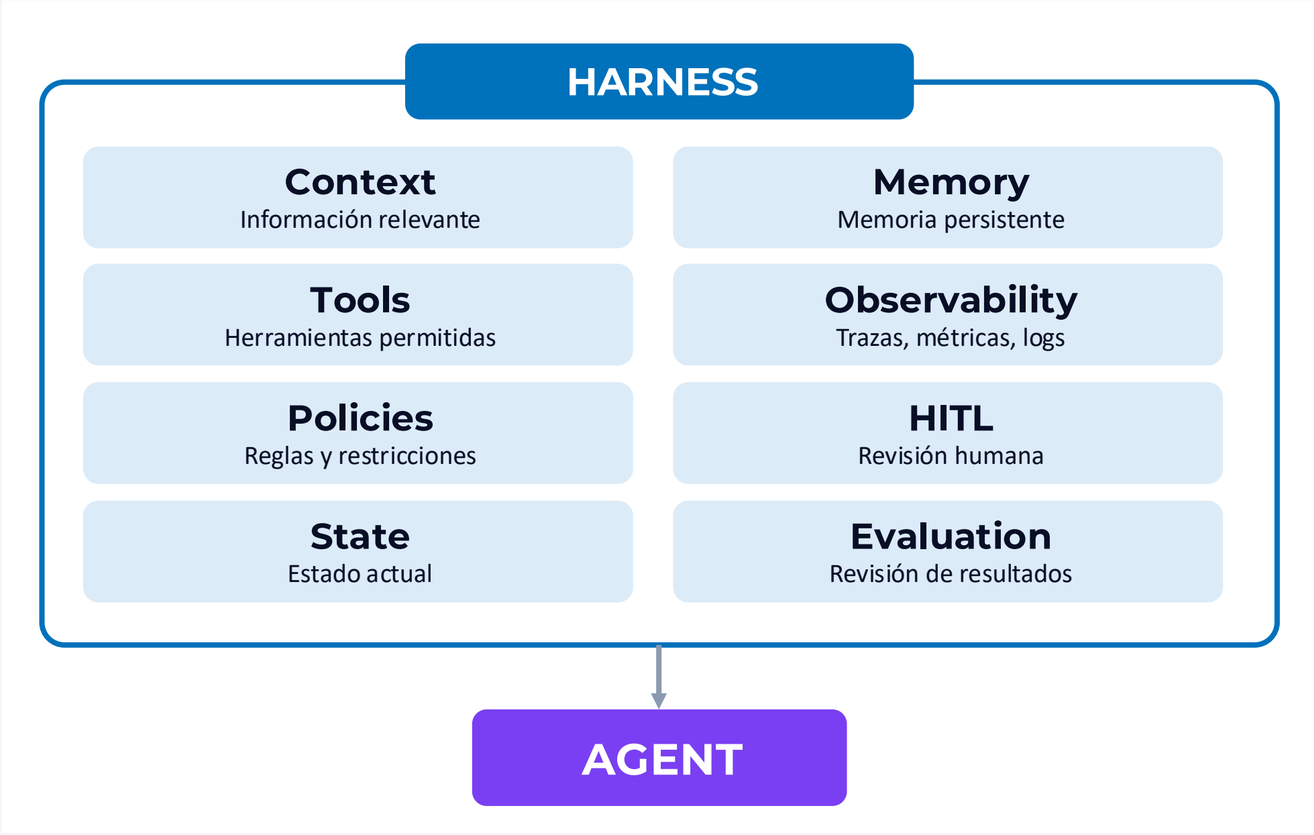
# Harness — Engineering

El modelo es una caja. Lo que **mide, decide, falla y cuesta** es el harness que lo envuelve — y es donde realmente se gana o se pierde la operación.

**EN ESTA SESIÓN** Modelo vs harness · cuatro métricas que importan · patrones de cache/retry/parallel/budget · cómo aterriza en BCP.

# Harness *Engineering.*

El entorno que controla al agente.




**NOTA**

**El Harness no es el agente:** es lo que envuelve al modelo, el entorno que habilita, limita, observa y controla al agente.

- **Habilita**
- **Limita**
- **Observa**
- **Controla**

**LO CLAVE** **El modelo razona; el harness decide con qué contexto, qué tools, qué reglas y bajo qué supervisión.**

© 2026 NTT DATA, Inc.

# El modelo no es el sistema. *El harness sí.*

El modelo responde. El harness **decide**: qué prompt darle, qué herramientas exponer, qué estado guardar, qué políticas aplicar, qué observar, cuánto gastar. Cambiar el modelo sin cambiar el harness es barajar naipes marcados.

EL MODELO

## LLM

**La caja que responde.**

Es una función pura con temperatura. Recibe tokens, devuelve tokens. **No tiene memoria**, no ejecuta, no sabe si una respuesta es correcta. Si lo cambias por otro modelo compatible, el flujo no debería romperse.

**Temperature tokens / slicense**

EL HARNESS

## 9

**Todo lo demás que vive alrededor.**

**Nueve piezas** que hoy miramos **cuatro**: **evaluación, observabilidad, coste y latencia**. Las demás aparecerán cuando sumemos CrewAI, LangGraph y guardrails.

**Prompts / tools / state / policies / memory / metrics**

**POR QUÉ IMPORTA** Dos equipos con el mismo modelo y harnesses distintos producen **sistemas con calidad, coste y riesgos radicalmente distintos**. El harness es lo que defendemos ante cumplimiento, lo que pagamos al CFO, lo que el cliente ve.

© 2026 NTT DATA, Inc.

# Cuatro métricas, *un tablero.*

Un agente sin métricas es una demo. Estas cuatro son el mínimo que el harness debe exponer antes de que el sistema toque un proceso real de BCP.

## 01 EVALUACIÓN
### ¿La respuesta es correcta?

Dataset de casos dorados, rúbrica o LLM-as-judge. Se corre en cada cambio de prompt, modelo o herramienta — igual que un test unitario.

## 02 OBSERVABILIDAD
### ¿Qué pasó, paso a paso?

Trazas por nodo: prompt, tool call, resultado, decisión del router. Sin trazas no hay auditoría ni diagnóstico de fallos.

## 03 COSTE
### ¿Cuánto cuesta cada caso?

Tokens de entrada y salida por nodo y por caso resuelto. El coste por caso — no por llamada — es el que se defiende ante negocio.

## 04 LATENCIA
### ¿Cuánto espera el usuario?

Tiempo total y por nodo. Un crew de cuatro agentes secuenciales suma latencias: hay que medir antes de prometer un SLA.

**EJERCICIO** Para el crew de la S1, define un umbral de aceptación por métrica antes de ejecutarlo.

© 2026 NTT DATA, Inc.

# Cinco pasos para *envolver al modelo.*

El harness no es el agente: es el entorno que lo **habilita, limita, observa y controla**. En el laboratorio lo construimos en este orden, sobre el flujo de riesgo de la S2.

**1**

### Configurar el harness

Prompt de sistema, modelo, límites de pasos y tokens.

**2**

### Definir herramientas

Solo las necesarias, con parámetros tipados y permisos mínimos.

**3**

### Aplicar políticas

Reglas deterministas: qué no puede hacer, cuándo escala.

**4**

### Agregar memoria

Estado persistente vía checkpointer; contexto acotado por turno.

**5**

### Evaluar resultados

Correr el set de casos y comparar contra los umbrales de la lámina anterior.

**ENTREGABLE DEL LABORATORIO**

Un agente que resuelve el caso de riesgo **con trazas visibles, coste medido y al menos una política que bloquea** una acción no permitida.

© 2026 NTT DATA, Inc.

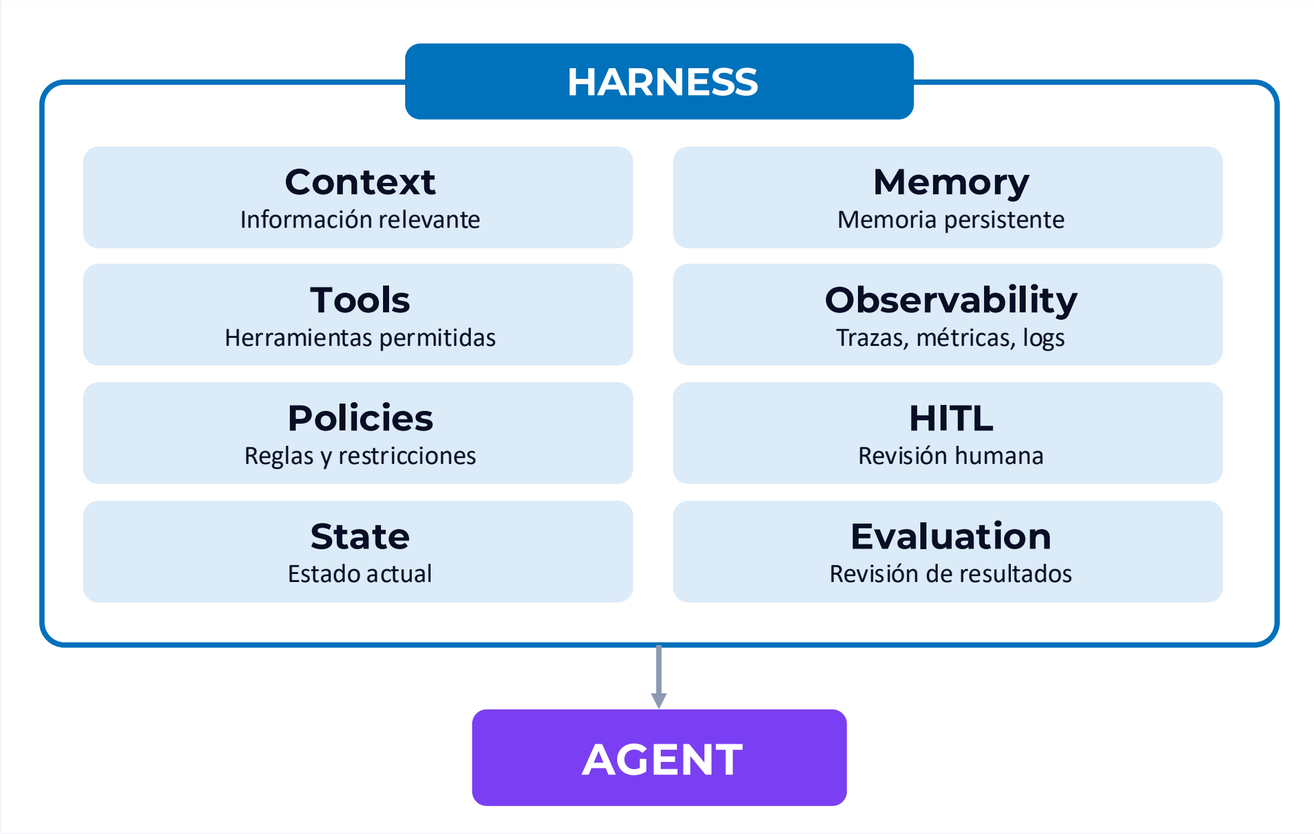
# Guardrails — Seguridad

Un guardrail es una capa de control que **valida o restringe** el comportamiento del sistema. En banca no es opcional: es la condición para salir a producción.

**Clave:** Cuatro tipos de guardrails · decisión PASS / BLOQUEAR / ESCALAR · flujo de seguridad extremo a extremo · Gate G2.

# Harness *Engineering.*

El entorno que controla al agente.




**NOTA**

**El Harness no es el agente:** es lo que envuelve al modelo, el entorno que habilita, limita, observa y controla al agente.

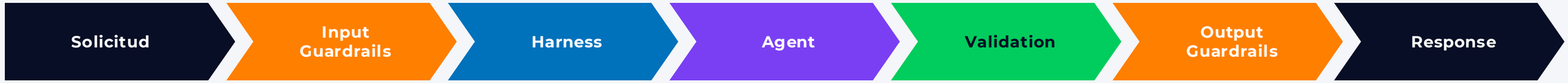
- **Habilita**
- **Limita**
- **Observa**
- **Controla**

**LO CLAVE** **El modelo razona; el harness decide con qué contexto, qué tools, qué reglas y bajo qué supervisión.**

© 2026 NTT DATA, Inc.

# Guardrails: *controla riesgos.*

Capas que validan o restringen el comportamiento del sistema.




*Los guardrails (naranja) actúan antes y después del agente; la validación confirma el resultado.*

**NOTA**

**Un GUARDRAIL:** es una capa de control que "VALIDA" o restringe el comportamiento del sistema.

**QUÉ HACE UN GUARDRAIL**

| | | |
|---|---|---|
| 1 | **Prevenir** | evita entradas o acciones riesgosas |
| 2 | **Detectar** | identifica violaciones o anomalías |
| 3 | **Bloquear** | detiene lo que no está permitido |
| 4 | **Corregir** | ajusta o reformula la salida |
| 5 | **Escalar a humano** | pasa la decisión a una persona |

**LO CLAVE** **Un buen guardrail previene, detecta y bloquea; y cuando duda, escala a una persona.**

© 2026 NTT DATA, Inc.

# Tipos de *Guardrails.*

Cuatro puntos del sistema donde se coloca un control.

ANTES DEL AGENTE

## Input Guardrails



- Contenido
- Intención / formato
- PII / prompt injection
- Tamaño

AL USAR HERRAMIENTAS

## Tool Guardrails



- Qué herramientas
- Parámetros
- Permisos
- Frecuencia / recursos

ANTES DE RESPONDER

## Output Guardrails



- Estructura
- Contenido
- PII / alucinaciones
- Políticas / formato

DURANTE LA EJECUCIÓN

## Runtime Guardrails



- Timeout
- Max pasos / tokens
- Presupuesto
- Loops / retries

**LO CLAVE** **Cada tipo cubre un momento distinto: entrada, herramienta, salida y ejecución. Se combinan.**

© 2026 NTT DATA, Inc.

Síntesis · La evolución del aprendizaje

# De agentes sueltos *a un sistema.*

Cada sesión respondió una pregunta. Juntas forman la arquitectura que vamos a construir en el hackathon.

**1**

## CrewAI

Multi-agent: roles, tareas y procesos.

***¿Cómo colaboran varios agentes?***

**2**

## LangGraph

State & graph: nodos, routers, checkpoints.

***¿Cómo controlo el flujo?***

**3**

## Harness

Context + tools + policies, medido en cuatro métricas.

***¿Cómo controlo el entorno?***

**4**

## Guardrails

Safety + validation, con HITL y Gate G2.

***¿Cómo evito riesgos?***

RESULTADO

**Un sistema de agentes seguro, controlado y escalable — listo para el hackathon del Día 5.**

© 2026 NTT DATA, Inc.


NTT DATA